

antigravity-proxy

Public

Watch 2

Fork 6

Star 14

main

1 Branch

0 Tags

Go to file

T

Add file

<> Code

12errh	feat(dashboard): redesign Sessions tab with card UI, checkboxes, and ...	717886b · 17 hours ago	93 Commits
.github	fix: npm ci fails on Windows because node-gyp cannot find V...	3 days ago	
docs	feat(models): add default model, custom mapping, per-mode...	18 hours ago	
proxy	feat(dashboard): redesign Sessions tab with card UI checkbo...	17 hours ago	
.gitignore	feat: add reasoning effort control, zen provider, cross-platfor...	last week	
AGENTS.md	feat(config): passthrough mode is now the default - external ...	yesterday	
CHANGELOG.md	docs: clarify why passthrough is default - external context gre...	yesterday	
CLAUDE.md	feat(config): passthrough mode is now the default - external ...	yesterday	
CODE_OF_CONDUCT.md	docs: add community files, fix CHANGELOG, update README	last week	
CONTRIBUTING.md	@	last week	
LICENSE	v1: Antigravity Proxy with NVIDIA + OpenRouter support	2 weeks ago	
README.md	docs: clarify why passthrough is default - external context gre...	yesterday	
SECURITY.md	feat(config): passthrough mode is now the default - external ...	yesterday	
agent-context.md	feat(router): close native vs external model parity gap across ...	yesterday	
start.ps1	feat: add reasoning effort control, zen provider, cross-platfor...	last week	
start.sh	@ fix: dashboard reads system env vars. fix start.sh mkdir and...	4 days ago	

README

Code of conduct

Contributing

MIT license

Security

Antigravity Proxy



Antigravity Proxy

Use any AI provider with Antigravity 2.0 — NVIDIA, OpenRouter, OpenAI, Groq, Anthropic, Google Gemini, OpenCode Zen, or 9+ local inference solutions.

passing

node >=20

license MIT

platform

Windows

macOS

Linux

The proxy intercepts Antigravity's Google Gemini API calls and translates them to the target provider's format. It supports:

- Multi-provider failover with retry + exponential backoff
- Per-model, per-provider routing via a matrix UI
- Reasoning effort control
- Real-time hot-reload of all config
- Advanced tool normalization and capability detection

Quick Start

Get the Repository

```
git clone https://github.com/12errh/antigravity-proxy.git
cd antigravity-proxy
```

All launcher scripts and source code are inside the cloned directory.

Prerequisites

- Antigravity 2.0 Desktop (Windows, macOS, or Linux)
- Node.js 20+
- Administrator/root privileges (port 443; can use 8443 without admin)
- At least one API key or local model server

Platform Support

Platform	Status	Notes
Windows 10/11	✅ Tested	start.ps1 handles cert install, port binding, and launch
macOS	⚠️ Untested	start.sh with best-effort cert trust. Report issues on GitHub.
Linux	⚠️ Untested	start.sh with best-effort cert trust. Report issues on GitHub.

TypeScript code is cross-platform. Only launcher scripts and TLS cert trust are platform-specific.

Launch

Windows (PowerShell as Administrator)

```
.\start.ps1
```

macOS / Linux

```
chmod +x start.sh
./start.sh
```

About

A TypeScript proxy that intercepts Antigravity's Gemini API and routes to NVIDIA or OpenRouter — chat, tools, browser automation, all working.

opensource

typescript

proxy

reverse-proxy

zen

local-llm

openrouter

ollama

llm-proxy

nvidia-nim

antigravity

Readme

MIT license

Code of conduct

Contributing

Security policy

Activity

14 stars

2 watching

6 forks

Report repository

Contributors 3

12errh

Rehan Mazhar

claude

Claude

openhands-agent

OpenHands

Languages

TypeScript

62.7%

HTML

34.3%

Shell

1.6%

PowerShell

1.1%

JavaScript

0.3%

Without admin (port 8443)

```
./start.sh --port 8443
```

Then open <http://localhost:4000> in your browser to configure providers, model mappings, and view live stats.

Key Features

Feature	Status	Description
Multi-provider failover	✓	Priority chain with exponential backoff
Per-model routing	✓	One row per model alias, one column per provider
Retry + backoff	✓	Configurable per-provider and global
Hot-reload config	✓	No restart needed for changes
Tool normalization	✓	Alias resolution, type coercion, default filling
Model capability detection	✓	Auto-detect reasoning, vision, tools
Provider optimizations	✓	Groq (image stripping), Zen/NVIDIA (reasoning effort)
Real-time dashboard	✓	Live logs, stats, config editor
Cost tracking	✓	Per-day, per-model, per-provider
Local discovery	✓	9+ inference solutions auto-detected
Full-text search	✓	Search requests, sessions, logs
Blocklist	✓	Provider, model, content filtering
Session comparison	✓	Side-by-side session analysis

How It Works



Per Request Pipeline

1. **TLS Intercept** — Receives Gemini API call on port 443
2. **Context Stripping** — Removes bulk context, injects compact `agent-context.md` reference
3. **Tool Normalization** — Resolves aliases, coerces types, fills defaults
4. **Model Resolution** — Routes through priority order with retry + backoff
5. **Capability Detection** — Auto-detects reasoning/vision/tool support
6. **Reasoning Effort** — Applies `reasoning_effort` if configured
7. **Translation** — Converts to target provider's API format
8. **Streaming** — Returns SSE response wrapped in Gemini format
9. **Logging** — Records to SQLite for cost tracking, history, sessions

Supported Providers

Cloud APIs

Provider	Type	Env Var	Notes
OpenRouter	Cloud API	<code>OPENROUTER_API_KEY</code>	300+ models in one key
NVIDIA NIM	Cloud API	<code>NVIDIA_API_KEY</code>	Free credits on signup
OpenAI	Cloud API	<code>OPENAI_API_KEY</code>	GPT-4o, o-series
Groq	Cloud API	<code>GROQ_API_KEY</code>	Ultra-fast LPU inference
Anthropic	Cloud API	<code>ANTHROPIC_API_KEY</code>	Claude 3.x / 4.x
Google Gemini	Cloud API	<code>GOOGLE_API_KEY</code>	Gemini 2.5 Pro/Flash
OpenCode Zen	Cloud gateway	<code>OPENCODE_API_KEY</code>	Claude, GPT, Gemini, Grok, Kimi, GLM — one key

Local Inference

Provider	Type	Notes
Ollama	Local	Auto-discovered on port 11434
vLLM	Local	Auto-discovered on port 8000
LM Studio	Local	Auto-discovered on port 1234
llama.cpp	Local	Auto-discovered on port 8080
text-generation-webui	Local	Auto-discovered on port 5000
TabbyAPI	Local	Auto-discovered on port 5000
LocalAI	Local	Auto-discovered on port 8080
LiteLLM	Local	Auto-discovered on port 4000
Aphrodite Engine	Local	Auto-discovered on port 8000

Models Tab — Matrix

The Models tab shows a matrix: one row per Antigravity model alias, one column per provider.



Features:

- Double-click any cell to open live model picker
- Column toggles to hide unused providers
- Quick-add buttons (+ Claude , + Gemini , + GPT)
- Empty cells use Default value
- Provider logos in filled cells

🧠 Model Options — Reasoning Effort

Some models support `reasoning_effort` to control thinking depth:

Model family	Levels
DeepSeek R-series	low, medium, high, max
NVIDIA stepfun	low, medium, high, max
OpenAI o-series	low, medium, high
Qwen Thinking, GLM Thinking, Kimi	low, medium, high

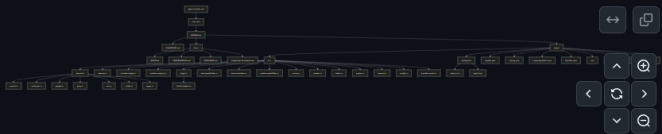
Auto-detection: The Model Options tab detects supported models and lets you set levels per model. Settings persist in `proxy/reasoning-effort.json` without restart.

⚙️ Config Tab — API Keys

Key	Provider
OPENROUTER_API_KEY	openrouter.ai/keys
NVIDIA_API_KEY	build.nvidia.com — free credits on signup
OPENAI_API_KEY	platform.openai.com/api-keys
GROQ_API_KEY	console.groq.com/keys
ANTHROPIC_API_KEY	console.anthropic.com/settings/keys
GOOGLE_API_KEY	aistudio.google.com/apikey
OPENCODE_API_KEY	opencode.ai/auth — one key for Claude, GPT, Gemini, Grok, Kimi, GLM

All keys are hot-reloaded from `.env` — no restart needed.

🏗️ Architecture



```
---

## 🌱 Environment Variables

'''env
# Provider priority (first = primary)
PROVIDER_PRIORITY=openrouter,nvidia,zen

# API keys
OPENROUTER_API_KEY=sk-or-v1-...
NVIDIA_API_KEY=nvapi-...
OPENAI_API_KEY=sk-...
GROQ_API_KEY=gsk-...
ANTHROPIC_API_KEY=sk-ant-...
GOOGLE_API_KEY=Aiza-...
OPENCODE_API_KEY=sk-...

# Ports
PROXY_PORT=443           # TLS port Antigravity connects to
API_PORT=4800            # Dashboard + REST API port

# Retry & fallback
PROXY_RETRIES=3          # Attempts per provider
PROXY_BACKOFF_MS=100     # Initial backoff (doubles each retry)

# Rate limiting (0 = unlimited)
RATE_LIMIT_GLOBAL=0
RATE_LIMIT_PROVIDER=0
RATE_LIMIT_WINDOW_MS=60000

# Logging
LOG_LEVEL=info           # debug | info | warn | error
LOG_MAX_SIZE_MB=10       # Rotate when reached
LOG_MAX_FILES=5          # Keep N rotated files
LOG_MAX_AGE_DAYS=30      # Delete files older than

# Dashboard auth (optional)
DASHBOARD_USER=admin
DASHBOARD_PASSWORD=your_password

# Failover webhook (optional)
FAILOVER_WEBHOOK_URL=https://hooks.example.com/webhook

# Workspace context hardening
WORKSPACE_CONTEXT_ENVELOPE=strict # off | loose | strict

# Inline context mode
CONTEXT_STRIP_MODE=passthrough    # passthrough (default) | strip
```

💡 Why Passthrough is the Default

We originally built an external context system (`agent-context.md` + `antigravity-context.ts`) to teach external models about Antigravity's tools. But after implementation, we discovered the external context grew to ~28K tokens — the exact same size as the native Antigravity context that Gemini receives.

Since both consume identical tokens, there's no benefit to stripping the native context and injecting our own. Passthrough simply forwards what AntigraVity already sends — simpler, no injection overhead, and external models understand it natively (validated: 18/20 tools working with MiMo-v2.5).

Future goal: Compress the external context to deliver the same native quality with fewer tokens, reducing the ~28K token overhead while maintaining full tool coverage.

Quick Links

- [Setup Guide](#)
- [Configuration Reference](#)
- [Developer Guide](#) ← **For adding providers & plugin development**
- [Antigravity v2 Protocol Analysis](#)
- [CHANGELOG](#)
- [Contributing](#)
- [Security Policy](#)

⚡ Development Quick Start

```
cd proxy
npm install          # Install dependencies
npm run build        # Compile TypeScript → dist/
npm run typecheck    # Type-check only
npm test             # Run tests
npm run dev          # Watch mode
```

New Providers: See [docs/DEVELOPER.md](#) for plugin architecture guide.

Run tests:

```
# All tests
npm test

# Filter by component
npx tsx test/run.ts plugin-architecture
npx tsx test/run.ts tool-translation
npx tsx test/run.ts model-discovery
npx tsx test/run.ts provider-adapters
```

📄 License

MIT — see [LICENSE](#).